

💡 تلخيص موسّع – الفصل السابع: العمليات (الدوال)

✅ ما هي العملية (الدالة)؟

العملية (Method) أو (Function) هي مجموعة من الأوامر تُكتب مرة واحدة وتُستخدم عدة مرات في البرنامج. هدفها الأساسي: تنفيذ مهمة معينة عندما نقوم باستدعائها (تشغيلها).

🔴 مثال واقعي: بدل ما تعيد وصفة الكعكة في كل مرة، تكتبها بورقة وتحفظها، وكل مرة بدك تعمل كعكة، ترجع للوصفة.

🏠 أهمية العمليات في البرمجة:

1. منع التكرار: (Duplication) بدل تكرار نفس الكود، نكتبه مرة داخل عملية.
2. تنظيم الكود: تقسيم البرنامج إلى أجزاء صغيرة.
3. سهولة الفهم والصيانة: أسهل على المبرمجين أن يفهموا البرنامج أو يعدلوا عليه.
4. إعادة الاستخدام: نستطيع استخدام العملية في أكثر من مكان.

🔄 كيف نكتب عملية؟

الصيغة العامة في: C#

(البارامترات) <اسم العملية> <نوع القيمة التي تُعيدها العملية>

```
{  
    أوامر //  
    return <قيمة>;  
}
```

🔴 مثال:

```
int Square(int x)  
{  
    return x * x;  
}
```

📦 البارامترات (Parameters)

هي القيم التي تُرسل للعملية عند استدعائها. يمكن إرسال:

- عدد واحد
- عدة أعداد
- أنواع مختلفة: أعداد صحيحة، نصوص، منطقية،...

مثال: 

```
void Greet(string name)
{
    Console.WriteLine("Hello, " + name);
}
```

return – القيمة المعادة 

بعض العمليات تُعيد ناتجًا (نتيجة) يمكننا استخدامه في مكان آخر بالبرنامج.

مثال: 

```
int Sum(int a, int b)
{
    return a + b;
}
```

العملية أعلاه ترجع مجموع عددين.

أنواع إرسال القيم: 

الطريقة	ماذا يحدث؟
by value	تُرسل نسخة من القيمة (التغيير لا يؤثر على الأصل).
by reference (ref)	تُرسل مرجع المتغير – التغيير يؤثر على الأصل.
باستخدام out	نستخدمه لإرجاع أكثر من قيمة من العملية.

مثال باستخدام out: 

```
void GetMinMax(int a, int b, out int min, out int max)
{
```

```
if (a > b) { max = a; min = b; }  
else { max = b; min = a; }  
}
```

🔗 مكس الاستدعاءات: (Call Stack)

عند استدعاء عملية، يُحفظ مكان العودة في "مكدس" خاص.
بعد تنفيذ العملية، يعود البرنامج تلقائيًا إلى السطر الذي استدعاه.

✅ مزايا العمليات:

- تنظيم الكود.
 - تسهيل الحلول المعقدة. (Top-Down Design)
 - فصل المهام المختلفة داخل البرنامج.
 - تقليل نسبة الخطأ في البرنامج.
-

📦 أمثلة تعليمية مناسبة للصف العاشر:

🎨 مثال 1: حساب عدد الأعداد فوق 90

```
int CountAbove90(int a, int b, int c)  
{  
    int count = 0;  
    if (a > 90) count++;  
    if (b > 90) count++;  
    if (c > 90) count++;  
    return count;  
}
```

✅ مثال 2: هل العمر بين 15 و 18؟

```
bool IsTeen(int age)  
{
```

```
return age >= 15 && age <= 18;
}
```

مثال 3: طباعة عدد من النجوم 🔄

```
void PrintStars(int count)
{
    for (int i = 0; i < count; i++)
        Console.Write("*");
}
```

أسئلة تتبع بسيطة للطلاب: 🧠

? تتبع 1:

```
int AddDigits(int num)
{
    int a = num % 10;
    int b = num / 10 % 10;
    int c = num / 100;
    return a + b + c;
}
Console.WriteLine(AddDigits(321));
```

ما هو المخرج؟ 📌

→ 1 + 2 + 3 = 6

? تتبع 2:

```
bool IsEven(int number)
{
    return number % 2 == 0;
}
```

```
Console.WriteLine(IsEven(7));
```

🚩 المخرج؟
→ False

🔍 تتبع 3: ref -

```
void MultiplyByTwo(ref int x)
{
    x = x * 2;
}
int a = 5;
MultiplyByTwo(ref a);
Console.WriteLine(a);
```

🚩 المخرج؟
(10 → لأن ref يغير القيمة الأصلية)

🔍 تتبع 4: out -

```
void GetData(out int x, out int y)
{
    x = 10;
    y = 20;
}
int a, b;
GetData(out a, out b);
Console.WriteLine(a + b);
```

🚩 المخرج؟
→ 30

🌟 نصائح للطلاب:

- العملية لا تعمل من تلقاء نفسها – يجب استدعاؤها!
 - إذا استخدمت `return` ، يجب أن تكتب نوع العملية: `(int, bool,...)`
 - انتبه لترتيب البارامترات عند الاستدعاء.
 - استخدم `ref` أو `out` فقط عند الحاجة لتغيير القيم الأصلية.
-